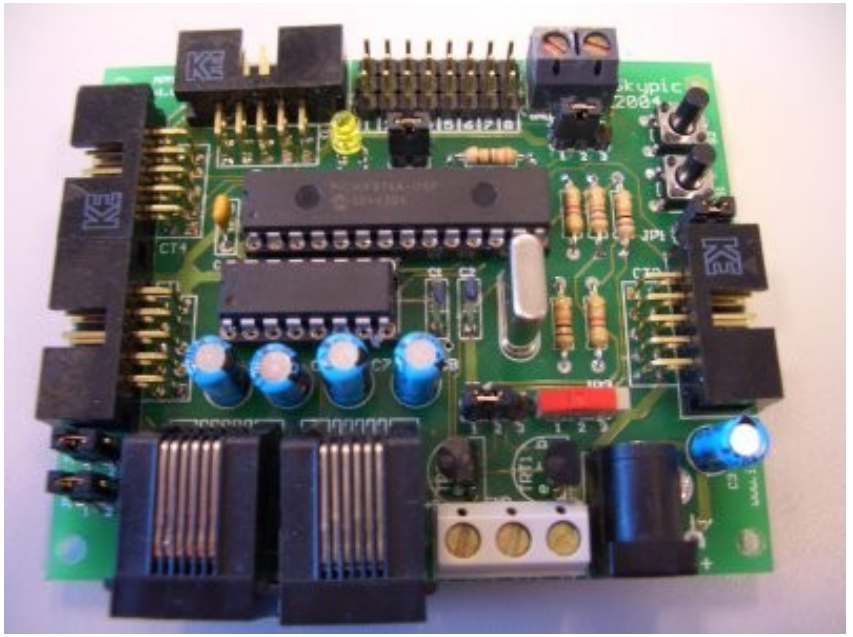
PIC 16F87X



Juan González

Escuela Politécnica Superior
Universidad Autónoma de Madrid

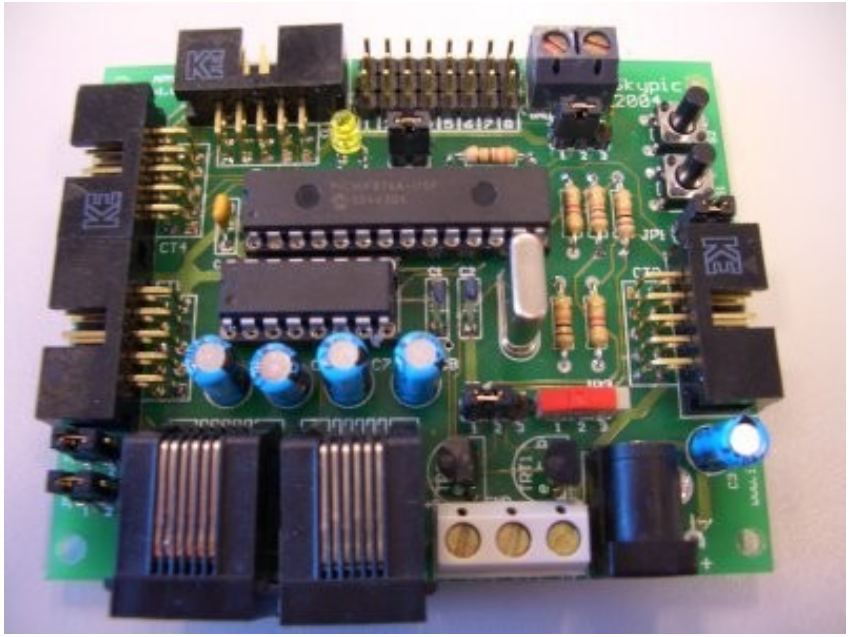
Andrés Prieto-Moreno

Flir Networked Systems

Ricardo Gómez

Flir Networked Systems

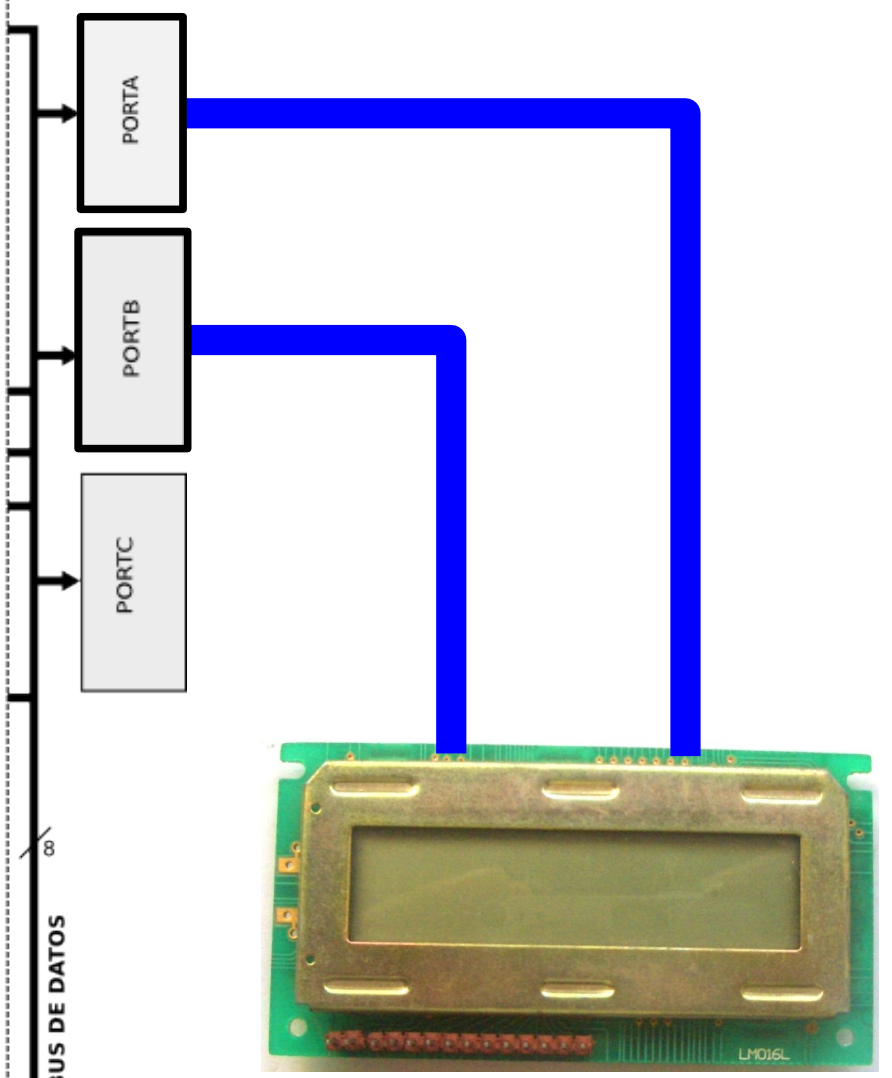
PIC 16F87X

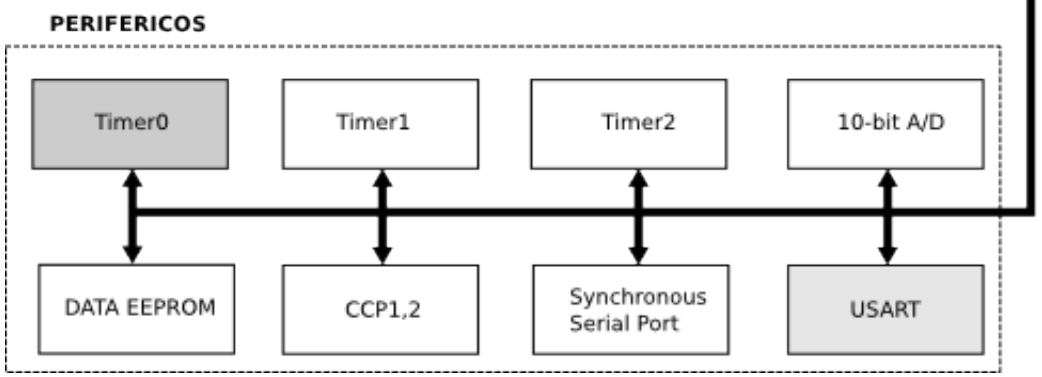


MÓDULO 5:

Displays de cristal líquido (LCD)

NUCLEO PIC16F876

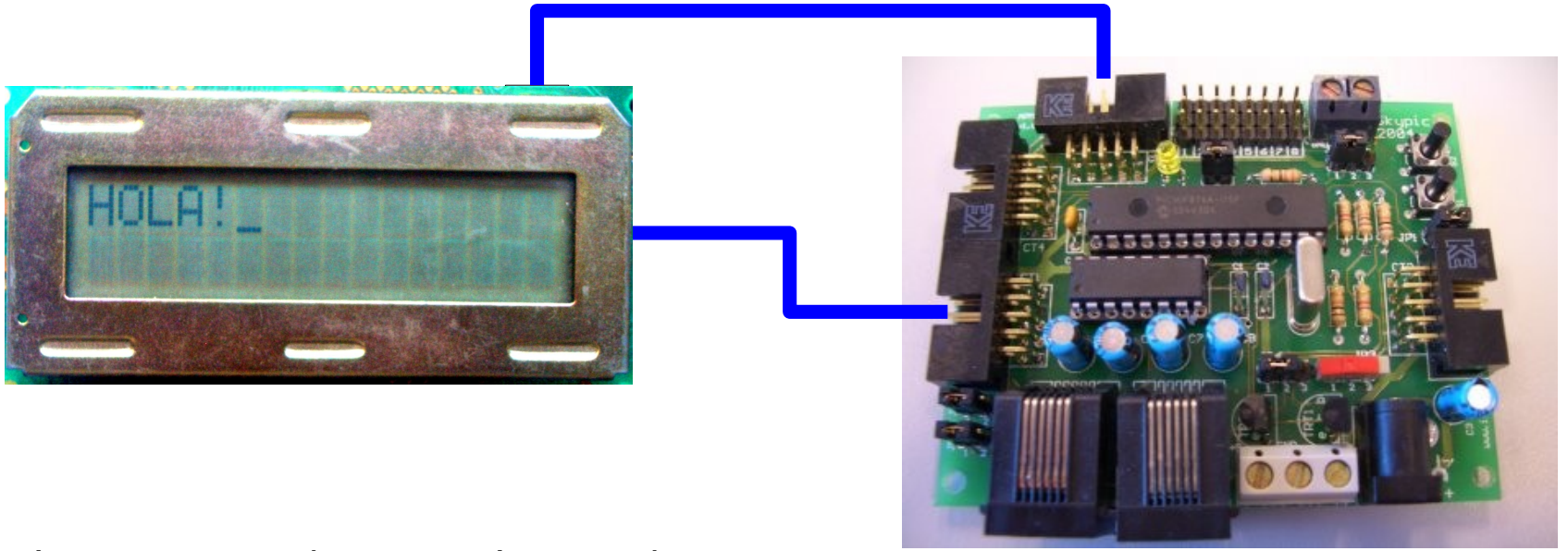




LCDs

Introducción

- Trabajaremos con un **LCD** de **2 líneas** y **16 caracteres** por línea
- Conectado a un **pic16f876a** (tarjeta Skypic) a través de los **puertos** A y B



- Bajo consumo (menor de 5mA)
- Tensión de alimentación: 5v
- Contraste regulable
- **Protocolo y comandos estandarizados**: Todos los fabricantes tienen el núcleo de control común, aunque añaden extensiones

Características (I)



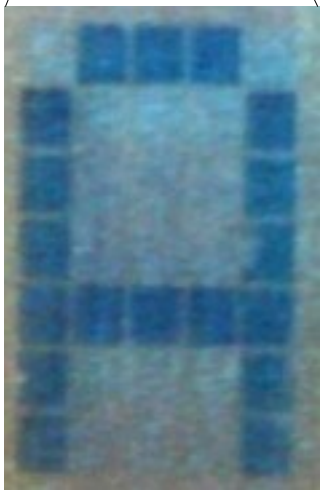
- **256 caracteres diferentes**
  - Tabla ASCII
  - 8 caracteres definibles por el usuario

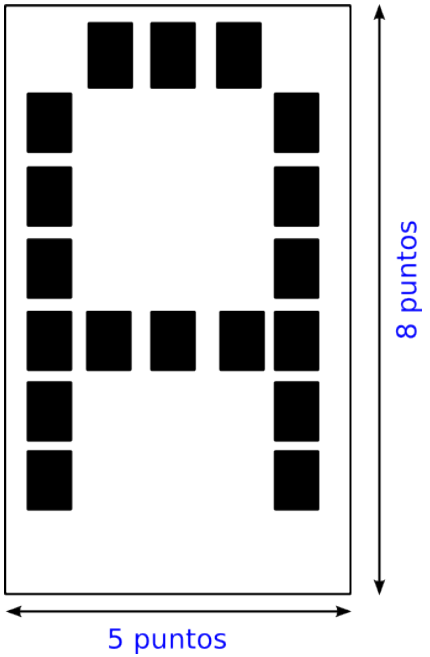
- **Display virtual:** 40 caracteres x 2 líneas



Características (II)

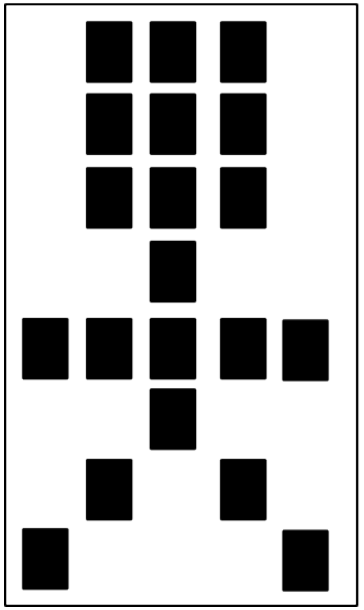






- Los caracteres tienen una resolución de **8x5 puntos**
- Para definir **nuevos caracteres** se utilizan 8 bytes, uno para cada fila. Sólo se usan los 5 bits menos significativos

Caracter definido por el usuario



Binario	Hexadecimal
01110	0x0E
01110	0x0E
01110	0x0E
00100	0x04
11111	0x1F
00100	0x04
01010	0x0A
10001	0x11

Memorias del LCD

- El LCD tiene **dos tipos de memorias RAM**:

Es RAM, por tanto **volátil**. Al apagar el LCD se pierde su contenido

DD RAM: *Display Data RAM*

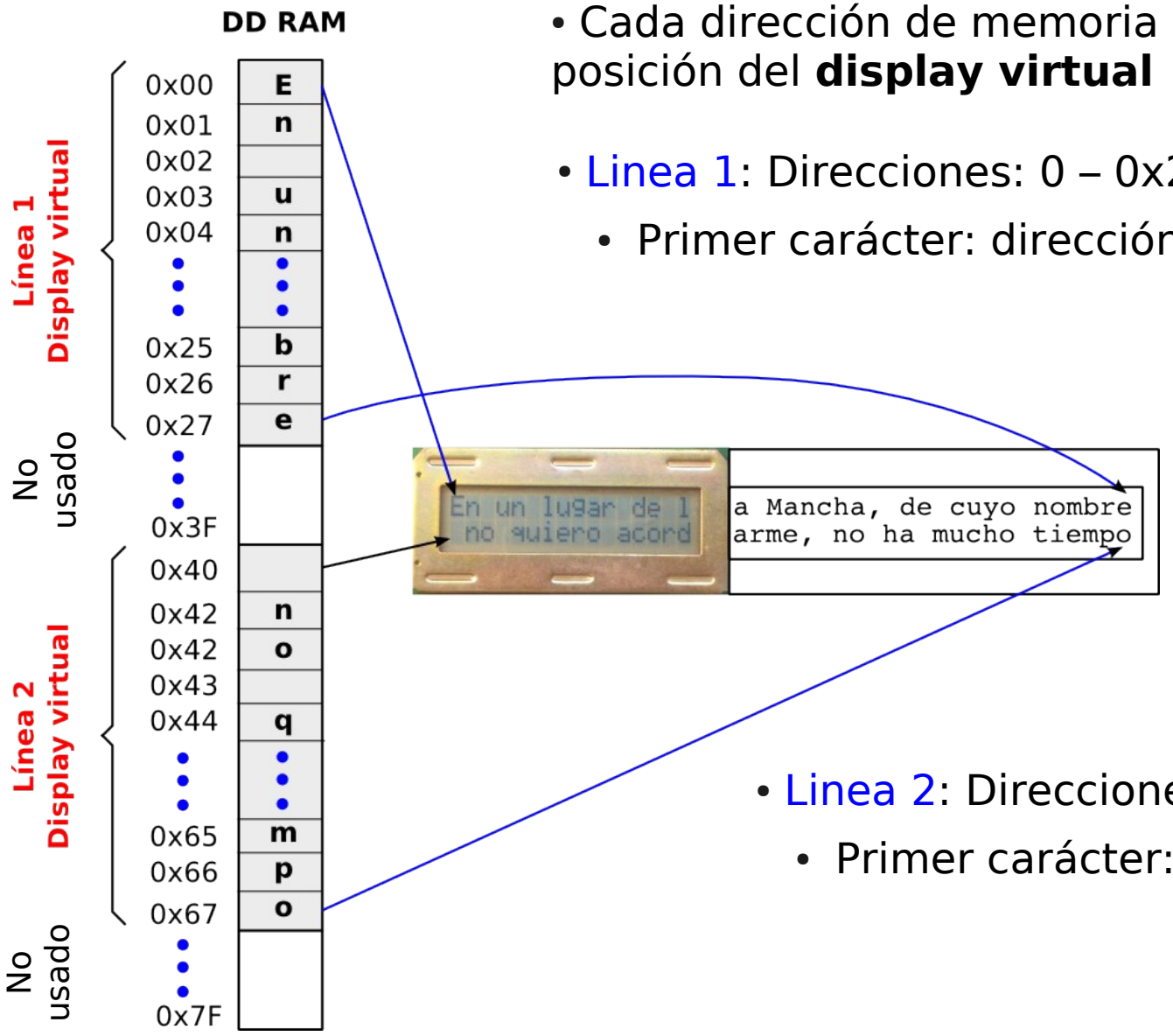
- Almacena los datos que se **muestran en el LCD**
- Direcciones de 7 bits (Capacidad de **128 bytes**)
- Rango de direcciones: 0 - 0x7F

CG RAM: *Character Generator RAM*

- Almacena los **caracteres definidos por el usuario**
- Direcciones de **6 bits** (Capacidad de **64 bytes**)
- Rango de direcciones: 0 - 0x3F

Memoria DD RAM

- Almacena todos los caracteres que se encuentran en **el display virtual**



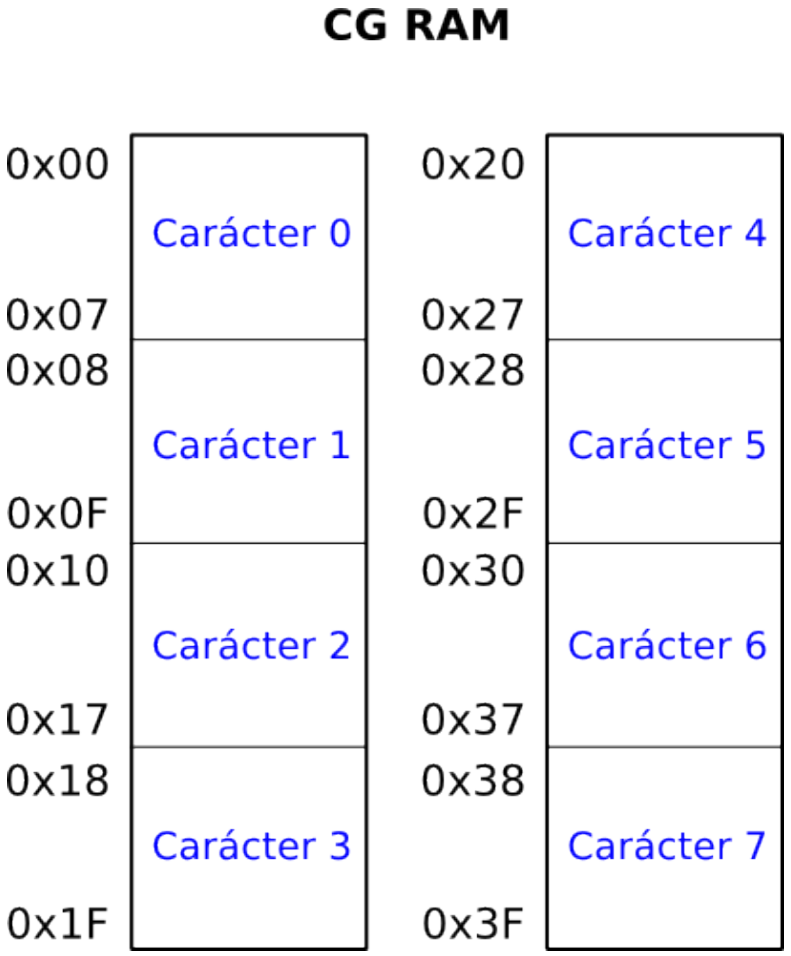
- Cada dirección de memoria asociada a una posición del **display virtual**

- Linea 1: Direcciones: 0 – 0x27
  - Primer carácter: dirección 0

- Linea 2: Direcciones: 0x40 – 0x67
  - Primer carácter: dirección 0x40

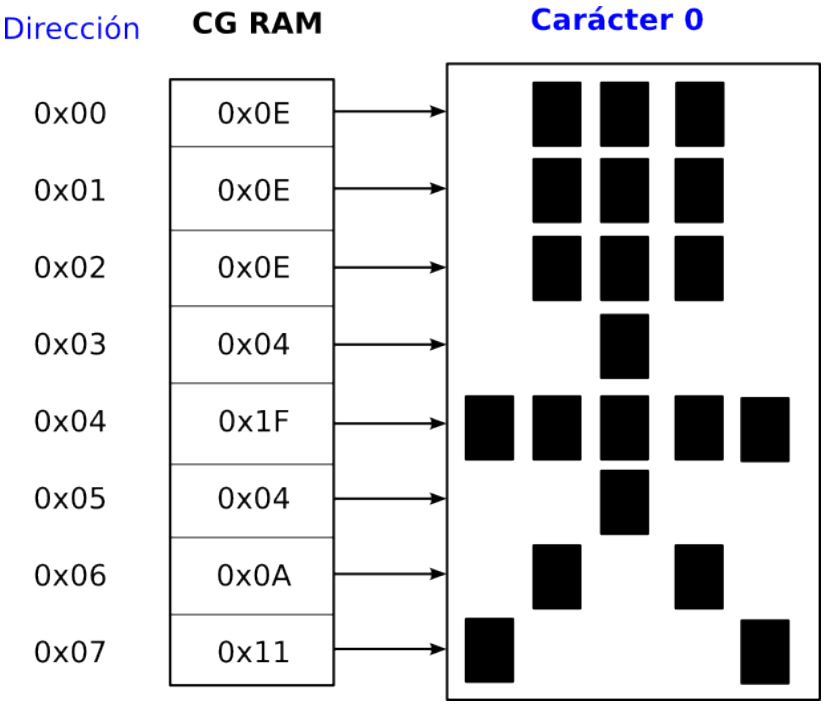
Memoria CG RAM

• Almacena la información para construir los caracteres definidos por el usuario



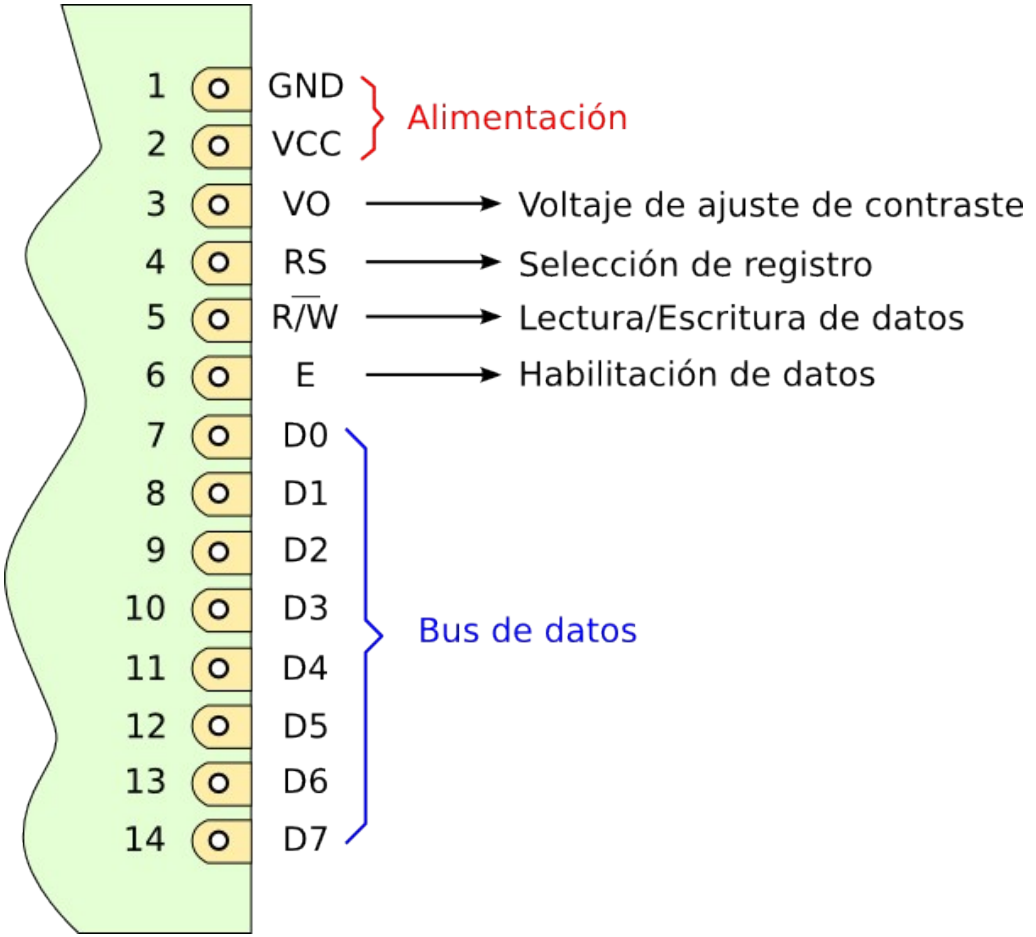
• **Ejemplo**: Definiendo el carácter 0

• Almacenar los siguientes datos en la CG RAM a partir de la dirección 0



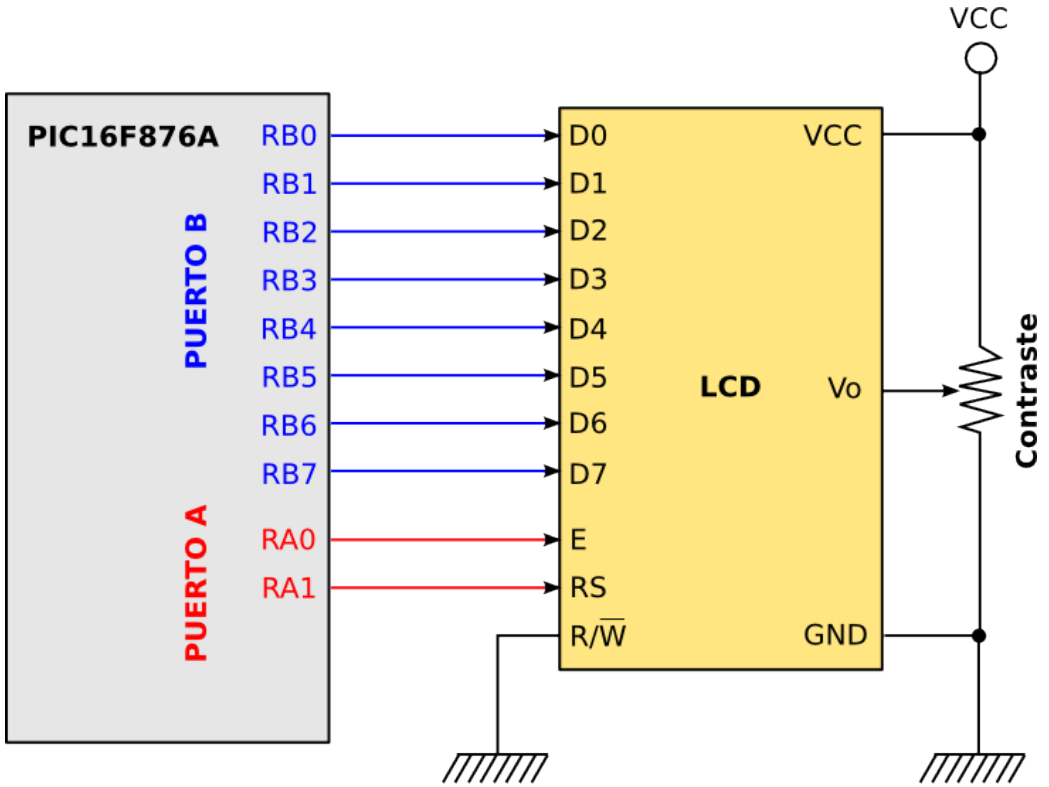
Patillaje del LCD

• Los LCDs tienen al menos estos **14 pines** (algunos tienen más, como por ejemplo el que permite la retroiluminación)



Conexión del LCD al PIC16F876A

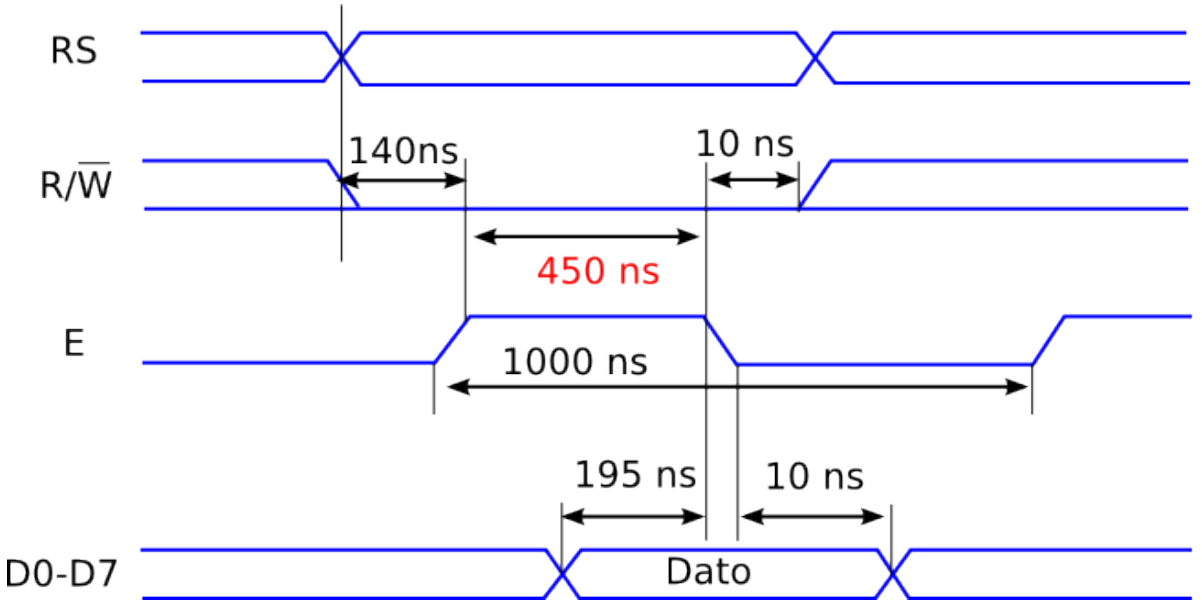
- La forma más sencilla es controlar el LCD usando un bus de datos de 8 bits y sin lectura de los sus registros internos
- En ese caso, son necesarios **10 pines del PIC**
- **8 pines** para el **bus de datos**
- 2 para las señales RS y E
- La señal R/W la ponemos a masa. No se van a leer datos del LCD



- Usaremos el puerto B completo, y dos bits del puerto A

Escritura en los registros del LCD

- Protocolo de envío de datos entre el PIC y el LCD
- Se indican los **tiempos mínimos**



Los datos se pueden enviar de la siguiente manera:

- Dar valores a RS y RW (nosotros dejamos RW siempre a cero)
- Escribir el dato
- Poner E a 1
- Poner E a 0

→ E deben permanecer al menos 500ns en cada estado

Registros en el LCD

El LCD dispone de **2 registros** donde escribir

Registro de Comandos

- El dato enviado se interpreta como un comando
- Se escribe en este registro cuando RS=0

Registro de Caracteres

- El dato enviado se interpreta como un carácter a ser impreso en el LCD
- Se escribe en este registro cuando RS=1

Comandos

	D7	D6	D5	D4	D3	D2	D1	D0
Borrar display (CLS)	0	0	0	0	0	0	0	1
Cursor al inicio (HOME)	0	0	0	0	0	0	1	0
Modo de funcionamiento	0	0	0	0	0	1	I/D	S
Control ON/OFF	0	0	0	0	1	D	C	B
Desplazamiento (SHIFT)	0	0	0	1	S/C	R/L	0	0
Modo de transferencia	0	0	1	DL	0	0	0	0
Acceso a CG RAM	0	1						
Acceso a DD RAM	1							

I/D → Contador de direcciones (1 incrementar, 0 decrementar)

S → Display (1 se desplaza, 0 quieto)

B → Parpadeo del cursor (1 activado, 0 desactivado)

C → Cursor (1 visible, 0 no visible)

D → Caracteres en pantalla (1 visibles, 0 no visibles)

S/C → Desplazamiento (1 display, 0 cursor)

R/L → Desplazamiento (1 derecha, 0 izquierda)

DL → Tamaño bus de datos (1-> 8 bits, 0-> 4 bits)

Software de manejo del LCD

- **Restricciones temporales**:
  - Las propias del protocolo
  - Los comandos tardan un tiempo máximo de **1.64ms** en ejecutarse
- Es necesario disponer de una **rutina de pausa**: usaremos creadas en los ejemplos del temporizador 0
- Esta rutina la llamaremos delay(). Se le pasan como parámetro la duración en centésimas de segundo
- Por simplicidad, no escribiremos el código de la rutina en las ejemplos mostrados en las siguientes transparencias

```
void delay(unsigned int duracion)
```

Ejemplo “hola mundo”

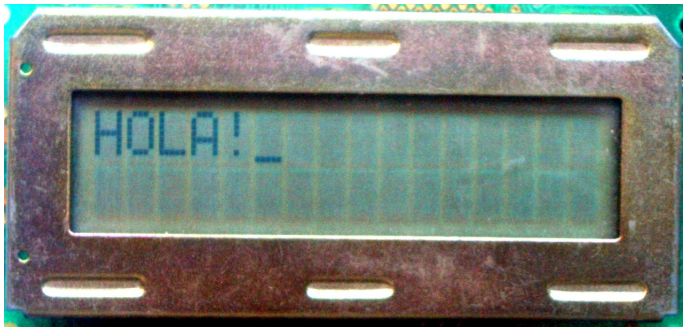
lcd-hola.c (parte I)

```
#include <pic16f876a.h>
#define RW  RA2
#define RS  RA1
#define E   RA0
#define INIT     0x38
#define ENCENDER 0x0E
#define HOME     0x02
#define CLS      0x01
void enable()
{
  E=1; E=1; E=1;
  E=0; E=0; E=0;
}
void lcd_cmd(unsigned char cmd)
{
  RS=0;
  PORTB=cmd;
  enable();
  delay(1);
}
```

Definir los pines que se usan para las señales de control

Algunos de los comandos usados en este ejemplo

Se inicializa el LCD y se escribe “HOLA!”



Función para actuar sobre la señal E y validar los datos enviados al LCD. Para garantizar que al menos permance 500ns en cada estado se repite tres veces cada comando

Función para enviar un comando al LCD

Seleccionar el registro de comandos

Enviar el comando por el puerto B

Validar el dato actual y que se escriba en el LCD

Antes de que se ejecute el siguiente comando al menos deben transcurrir 1.64ms

Ejemplo “hola mundo” (II)

lcd-hola.c (parte II)

```
void lcd_write(unsigned char car)
{
  RS=1;
  PORTB=car;
  enable();
  delay(1);
}

void main(void)
{
  TRISB=0x00;
  TRISA=0xF8;
  ADCON1=0x06;
  T0CS=0; PSA=0;
  PS2=1; PS1=1; PS0=1;
  (...)
```

Función para escribir un carácter en el LCD

Seleccionar registro de caracteres

Poner el carácter en el puerto B

Validar el dato, para que se escriba en el LCD

Antes de un cierto tiempo no se puede volver a escribir otro carácter

Configurar el puerto B para salida

Configurar RA0, RA1 y RA2 como salidas

Configurar puerto A como E/S digital

Configurar el timer 0 para que las rutinas delay funcione correctamente

Ejemplo “hola mundo” (III)

lcd-hola.c (parte III)

```
    delay(1);
    RW=0; E=0;

    lcd_cmd(INIT);
    lcd_cmd(0x06);
    lcd_cmd(ENCENDER);
    lcd_cmd(HOME);
    lcd_cmd(CLS);

    lcd_write('H');
    lcd_write('O');
    lcd_write('L');
    lcd_write('A');
    lcd_write('!');

    while(1);
}
```

Pausa inicial. El display tarda un tiempo en inicializarse

Inicializar las señales del bus de control

Secuencia de inicialización. Primero se establece el ancho del bus de datos a 8 bits

Establecer el modo de funcionamiento: el display permanece “quieto”. El contador de direcciones se incrementa

Display activado (los caracteres serán visibles). Cursor NO parpadea y cursor es visible

Display activado (los caracteres serán visibles). Cursor NO parpadea y cursor es visible

Llevar el cursos a la posición inicial

Borrar el LCD

Escribir caracteres en el LCD!!!!

LCD y puerto serie (I)





- Vamos a “conectar” el LCD al **puerto serie**
- Desde el terminal de comunicaciones del PC enviaremos caracteres que se mostrarán en el LCD
- Es muy útil para hacer pruebas y ver visualmente el funcionamiento del LCD

Esto permite que nuestros programas que se ejecuten en el PC puedan también sacar información por el LCD

LCD y puerto serie (II)

lcd-term.c

```
#include <pic16f876a.h>
#include "sci.h"

  (...)
void main(void)
{
  unsigned char c;
  (...)
  sci_conf();

  while(1) {
    c=sci_read();
    lcd_write(c);
  }
}
```

Usaremos la librería del puerto serie que ya conocemos

Las definiciones y resto de funciones son las mismas que en el ejemplo anterior

La inicialización de los puertos, timer0 y LCD es igual que en el ejemplo anterior

Inicializar puerto serie

Bucle principal: muy sencillo. Todo lo que se recibe por el puerto serie se escribe en el LCD

Probando los comandos del LCD (I)

- Para jugar con los comandos del lcd y entender qué es lo que hacen, haremos un programa que nos envíe un **menú de opciones** por el **puerto serie**
- Los caracteres tecleados se imprimen en el LCD
- Las opciones del menú nos permitirán ejecutar **diferentes comandos**

```
Menu
----
1.- CLS
2.- HOME
3.- Mover display al imprimir
4.- Display quieto al imprimir
5.- Cursor a la izquierda
6.- Cursor a la derecha
Opcion:
```

Probando los comandos del LCD (II)

lcd-menu.c

```
(...)
void main(void)
{
  (...)
  menu();

  while(1) {
    c=sci_read();
    switch(c) {
      case '1':
        lcd_cmd(CLS);
        break;
      case '2':
        lcd_cmd(HOME);
        break;
      case '3':
        lcd_cmd(0x07);
        break;
      case '4':
        lcd_cmd(0x06);
        break;
```

```
      case '5':
        lcd_cmd(0x10);
        break;
      case '6':
        lcd_cmd(0x14);
        break;
      default:
        lcd_write(c);
    }
  }
}
```

Hasta aquí es igual que en el ejemplo anterior

Esta parte inicial también es igual

Sacar el menu por el puerto serie. Igual que en ejemplo sci-menu, pero con otras opciones

Según el carácter recibido ejecutar un comando diferente

Si el carácter recibido no es niguna de las opciones anteriores, entonces imprimirlo en el LCD

Ejercicio:

- Modificar el programa lcd-menu para probar el resto de comandos

Caracteres definidos por el usuario (I)

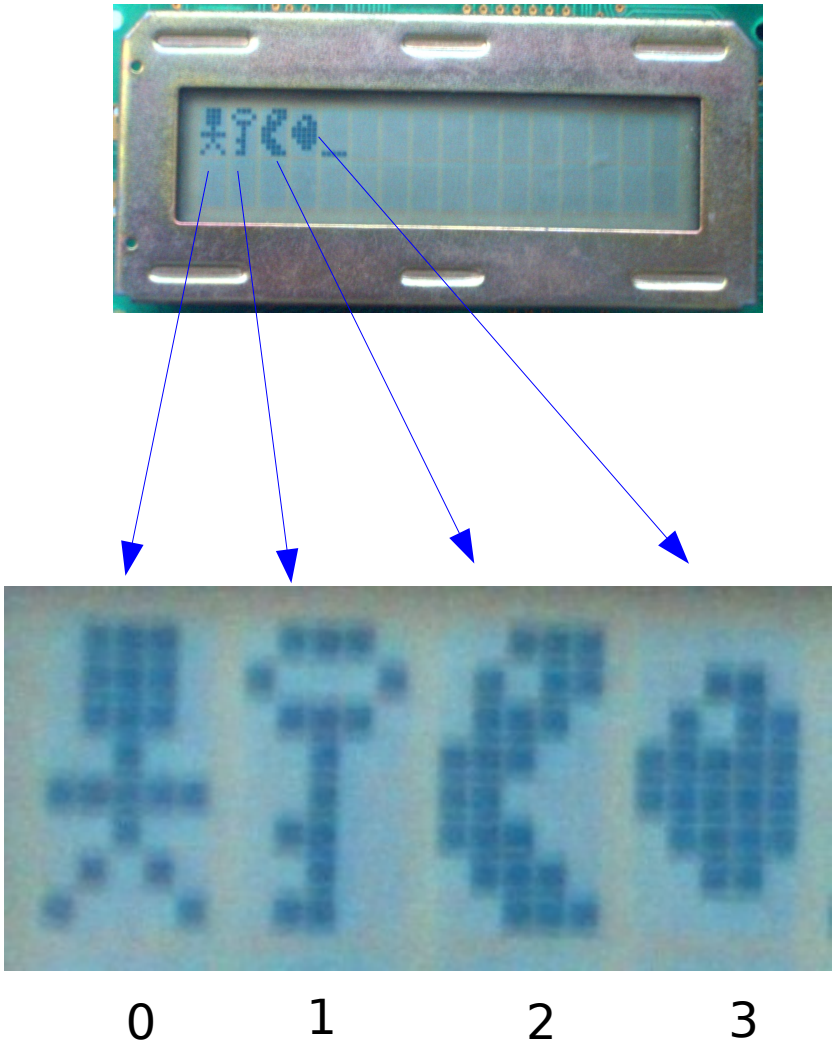
• Para crear caracteres nuevos tenemos que escribir en la **CG RAM**.

• Si empezamos definiendo el carácter 0, hay que acceder a la dirección 0 (El comando a enviar será el 0x40).

• A continuación enviamos los **8 bytes** que conforman las 8 líneas del carácter

• Para imprimir el nuevo carácter, hay que acceder a la **DD RAM** (por ejemplo a la dirección inicial 0) e imprimir el carácter 0

Caracteres definidos de ejemplo :



Caracteres definidos por el usuario (II)

lcd-usuario.c (parte I)

```
(...)
void main(void)
{
(...)
lcd_cmd(0x40);
lcd_write(0x0E);
lcd_write(0x0E);
lcd_write(0x0E);
lcd_write(0x04);
lcd_write(0x1F);
lcd_write(0x04);
lcd_write(0x0A);
lcd_write(0x11);

lcd_write(0x0E);
lcd_write(0x11);
lcd_write(0x0E);
lcd_write(0x04);
lcd_write(0x04);
lcd_write(0x0C);
lcd_write(0x04);
lcd_write(0x0C);
```

Hasta aquí es igual que en en los ejemplos anteriores

Acceder a la dirección 0 de la CG RAM

Estableder las 8 líneas del carácter 0. Es un monigote

Estableder las 8 líneas del carácter 1. Es una llave. No hace falta enviar un comando con la dirección, ya que el carácter 1 está a continuación del 0

Caracteres definidos por el usuario (III)

lcd-usuario.c (parte II)

```
lcd_write(0x07);
lcd_write(0x0B);
lcd_write(0x0E);
lcd_write(0x1C);
lcd_write(0x18);
lcd_write(0x1C);
lcd_write(0x0E);
lcd_write(0x07);

lcd_write(0x00);
lcd_write(0x06);
lcd_write(0x0B);
lcd_write(0x1F);
lcd_write(0x1F);
lcd_write(0x0F);
lcd_write(0x06);
lcd_write(0x00);
```

Estableder las 8 líneas del carácter 2. Es un comecocos con la boca abierta

Estableder las 8 líneas del carácter 3. Es un comecocos con la bocacerrada

```
    lcd_cmd(0x80);
    lcd_write(0);
    lcd_write(1);
    lcd_write(2);
    lcd_write(3);


    while(1);
}
```

Acceder a la dirección 0 de la DD RAM para empezar a imprimir

¡Imprimir los 4 nuevos caracteres creados!

Ejercicio final del LCD

- **Hacer una animación** en el LCD. Un carácter tiene que partir de la posición inicial y moverse en horizontal, en ambos sentidos. Primero hacia la derecha y cuando alcance el borde hacia la izquierda

Mejoras:

- **Animar el carácter**. Por ejemplo hacer que el comecocos abra y cierra la boca según se desplaza (pero os podéis definir los caracteres que queráis ;-)
- **Añadir sonido**. Por ejemplo que cuando llegue a uno de los extremos emita un pitido.